

PDF Editor - Design Choices

The project uses a Flutter client with a Python FastAPI backend.

1. PyMuPDF is used for text extraction, PDF generation, and watermarking because it provides a single dependency with Unicode and HTML-layout support.
2. Translation uses deep-translator, calling Google Translate first and MyMemory as a fallback when the free Google endpoint is unavailable or rate-limited.
3. Dio handles multipart HTTP requests in Flutter, while file_picker, path_provider, and open_filex support file selection and result handling.
4. Generated PDFs embed both Noto Sans and Noto Sans Bengali fonts, loaded through PyMuPDF's font archive and referenced with CSS @font-face.
5. Noto Sans because:
 - **Noto Sans** covers Latin text and many common symbols consistently.
 - **Noto Sans Bengali** is designed specifically for Bangla script, including conjuncts, vowel signs, and other complex character combinations.
6. The same font configuration is used for watermark text, so Bangla can also be used in watermarks.

Limitations

- ❖ Translated PDFs are rebuilt on A4 pages.
- ❖ The original PDF's layout, images, fonts, tables, and exact page structure are not preserved.
- ❖ Translation requires internet access.
- ❖ Scanned PDFs without an extractable text layer cannot be translated because the project does not include OCR.
- ❖ Uploads are limited to 20 MB.